



RAJALAKSHMI ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

CS23432 - Software Construction.

Laboratory Record Note Book

NAME Hannah Panselvi. P

BRANCH B.Tech Information Technology

UNIVERSITY REGISTER No. 2116241001069

COLLEGE ROLL No. 241001069

SEMESTER IV

ACADEMIC YEAR 2025 - 2026



**RAJALAKSHMI
ENGINEERING COLLEGE**

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME Hannab. Ponselvi. P.

ACADEMIC YEAR 2025 - 2026 SEMESTER IV BRANCH IT

UNIVERSITY REGISTER No.

2116241001069

Certified that this is the bonafide record of work done by the above student in the

CS23432 - Laboratory during the year 2025 - 2026
Software Construction

Signature of Faculty - in - Charge

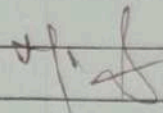
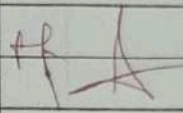
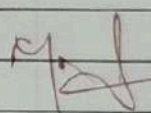
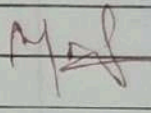
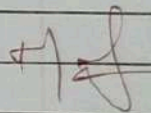
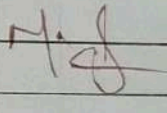
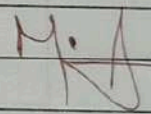
Submitted for the Practical Examination held on12.05.2026..

Internal Examiner

External Examiner

I N D E X

NAME: Hannah Porsehi P STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: SC OBSERVATION

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	21/4/26	AZURE DEVOPS ENVIRONMENT SETUP	3	
2.	5/2/26	AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT	5	
3.	26/2/26	SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING	7	
4.	5/3/26	SPRINT PLANNING	9	
5.	12/3/26	POKER ESTIMATION	13	
6.	19/3/26	DESIGNING CLASS AND SEQUENCE DIAGRAM FOR PROJECT ARCHITECTURE	15	
7.	2./4/26	DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE	19	
8.	16/4/26	TESTING - TEST PLANS AND TEST CASES.	25	
9.	16/4/26	LOAD TESTING AND PIPELINES.	29	
10.	30/4/26	GITHUB : PROJECT STRUCTURE & NAMING CONVENTIONS	33	

Exp. 1
21/1/26

AZURE DEVOPS ENVIRONMENT SETUP

AIM:

To setup and access the Azure DevOps environment by creating an organization through the Azure portal to manage the login & Authentication system.

OBSERVATION:

Access Portal:

Open a web browser and sign in to the Azure portal using valid Microsoft account credentials. Search Search Dev Ops:

Navigate to the Azure home page and login "Azure DevOps Organizations" using the global search bar.

Organization Creation:

Click the "My Azure DevOps Organization" link to initiate the creation of a new organization for the project.

Account Verification:

Confirm that the organization is correctly linked to your student or professional Microsoft directory.

Landing Page:

Once created, ensure you are directed to the Azure DevOps Organization Homepage to begin project configuration.

RESULT:

Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

Ex. 2,
5/2/26

AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT

AIM:

To set up an Azure DevOps project for efficient collaboration and to manage the Role Assignment features.

OBSERVATION:

Initiate Project:

On the organization home page, click the "New Project" button to start the setup.

Define Identity:

Enter a unique project name (e.g., "Student Academic System") and provide a detailed description of the role management scope.

Privacy Controls:

Set the project visibility to "Private" to ensure that sensitive role assignments are only accessible to authorized administrators.

Access Boards:

Navigate to the "Boards" section from the left-hand menu to manage agile work items.

User Story Creation:

Use the "Add Work Item" dropdown to select "User Story" and document the specific requirement for admin role assignments.

RESULT:

Successfully created an Azure DevOps project with active user story management and agile workflow setup.

AZURE DEVOPSAIM:

Create and Organize Epics, Features, and User Stories for effective project planning.

OBSERVATION:

1. Open Azure DevOps Portal and navigate to the project dashboard.
2. Select Boards and then work items.
3. Create Epics for major functional modules.
4. Add Features under Epics and User Stories Under Features
5. Fill in details such as Title, Description, Assigned To, and Priority. Establishing the hierarchy by linking Epic to Features and Features to User Story.
6. Verify all created items in the Backlogs section.

PROJECT WORK ITEMS:

1. Epic: Login and Authentication Management.
Features: User Login System.
User Story: secure login access
2. Epic: User and Role Management.
Feature: Role Assignment
User Story: Assign Roles for access Control.
3. Epic: Grade Evaluation Engine.
Features: GPA Calculation.
User Story: Track performance via GPA.
4. Epic: Notification and Alert Management.
Features: Deadline Alerts.
User Story: Reminders for Submissions.

3. Epic: Activity and Academic Tracking

Features: Attendance Tracking

User Story: Monitor student attendance.

RESULT :

Epics, Features, and User stories created and
Organized in Azure DevOps for effective project
management.

AIM :

To assign User stories to different Sprints for effective planning and smooth project development.

OBSERVATION :-

Opened the Azure DevOps portal and navigated to the project dashboard.

Selected Boards → Sprints from the menu.

Created multiple sprints (sprint 1, sprint 2, sprint 3, sprint 4) and defined their durations.

Accessed the Backlogs section to view all User Stories.

Assigned User stories to respective sprint based on priority and functionality using drag-and-drop.

Distributed tasks among team members and updated their status.

Verified the sprint backlog to ensure proper work allocation and saved the changes.

SPRINT PLANNING:

* Sprint 1 : User Login System, Role Assignment
Focus : Basic setup and authentication.

* Sprint 2 : GPA Calculation

Focus : Academic Performance Module.

* Sprint 3 : Deadline Alerts

Focus : Notification and reminder system.

* Sprint 4 : Attendance Tracking

Focus : Activity and academic tracking.

RESULT :-

User stories were assigned successfully. Sprint planning improves organization and productivity.

Ex. 5
12/3/26

POKER ESTIMATION

AIM :-

To conduct Poker Estimation for the Attendance Tracking user stories to quantify the effort required for faculty monitoring.

OBSERVATION :

Collaborative Review :

Assemble the team to review the "As a faculty, I want to track attendance" user story.

Effort Discussion :

Discuss the technical complexities of real-time attendance logging and data storage.

Voting Process :

Use Poker Estimation cards to provide independent estimates of the story's difficulty.

Consensus Reached :

Align on a final value (Story Points) that reflects the consensus of the development team.

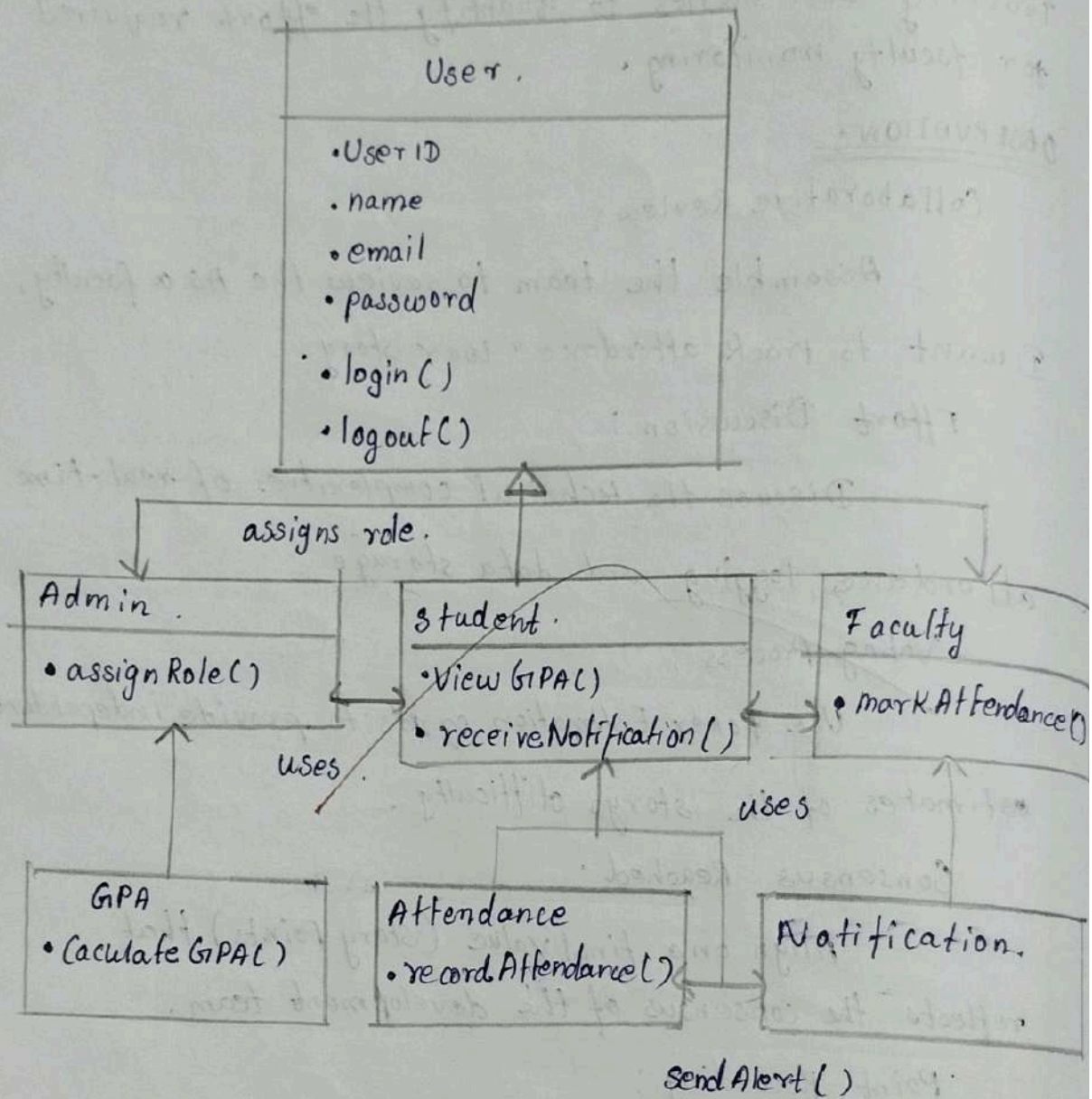
Point Recording :

Update the User Story in Azure DevOps with the agreed-upon story points to assist in future velocity tracking.

RESULT :

The estimation and story points for the Academic Tracking system have been successfully recorded using the Poker Estimation method.

Class Diagram



AIM:

To design Class and Sequence Diagrams for the given Project to represent system structure and interactions.

OBSERVATION:

Analyze project requirements and identified key entities like User, Admin, Student, Faculty, GPA, and Attendance.

Designed a Class Diagram with classes, attributes, methods, and relationships (inheritance, association).

Created a sequence Diagram to show interactions such as logging and Attendance.

Ensured clear representation of system structure and behaviour.

Class Diagram Description:

* User Class : Attributes - UserID, name, email, password ; Methods - login(), logout().

* Admin Class (inherits User) : assign Role()

* Student Class (inherits User) : view GPA()

* Faculty Class (inherits User) : mark Attendance()

* GPA class : calculate GPA()

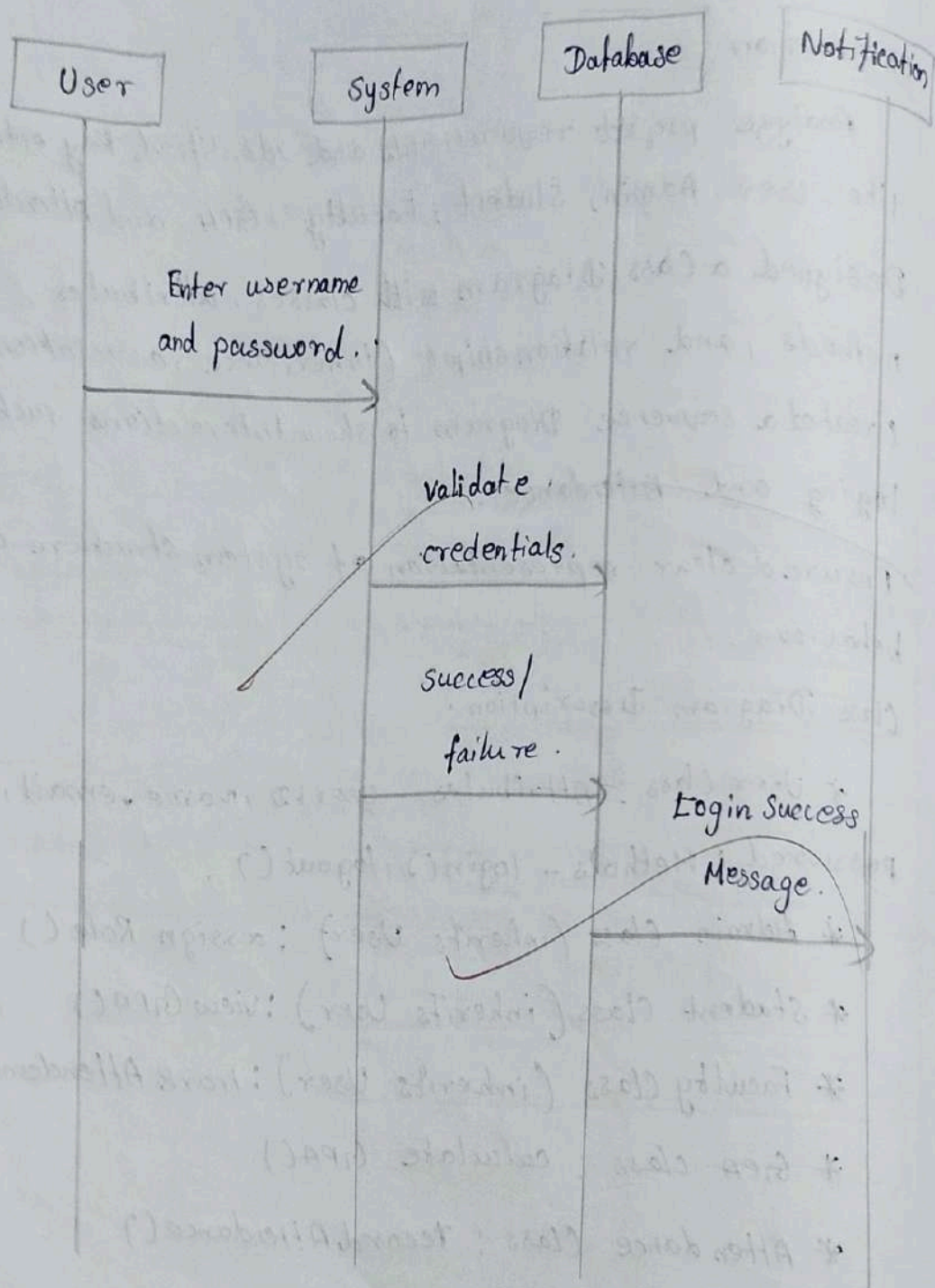
* Attendance Class : record Attendance()

SEQUENCE DIAGRAM DESCRIPTION:

* Login Process : User enters credentials → system validates - database verifies → access granted or denied.

* Attendance Tracking : Faculty logs in → selects students → mark attendance → system stores data

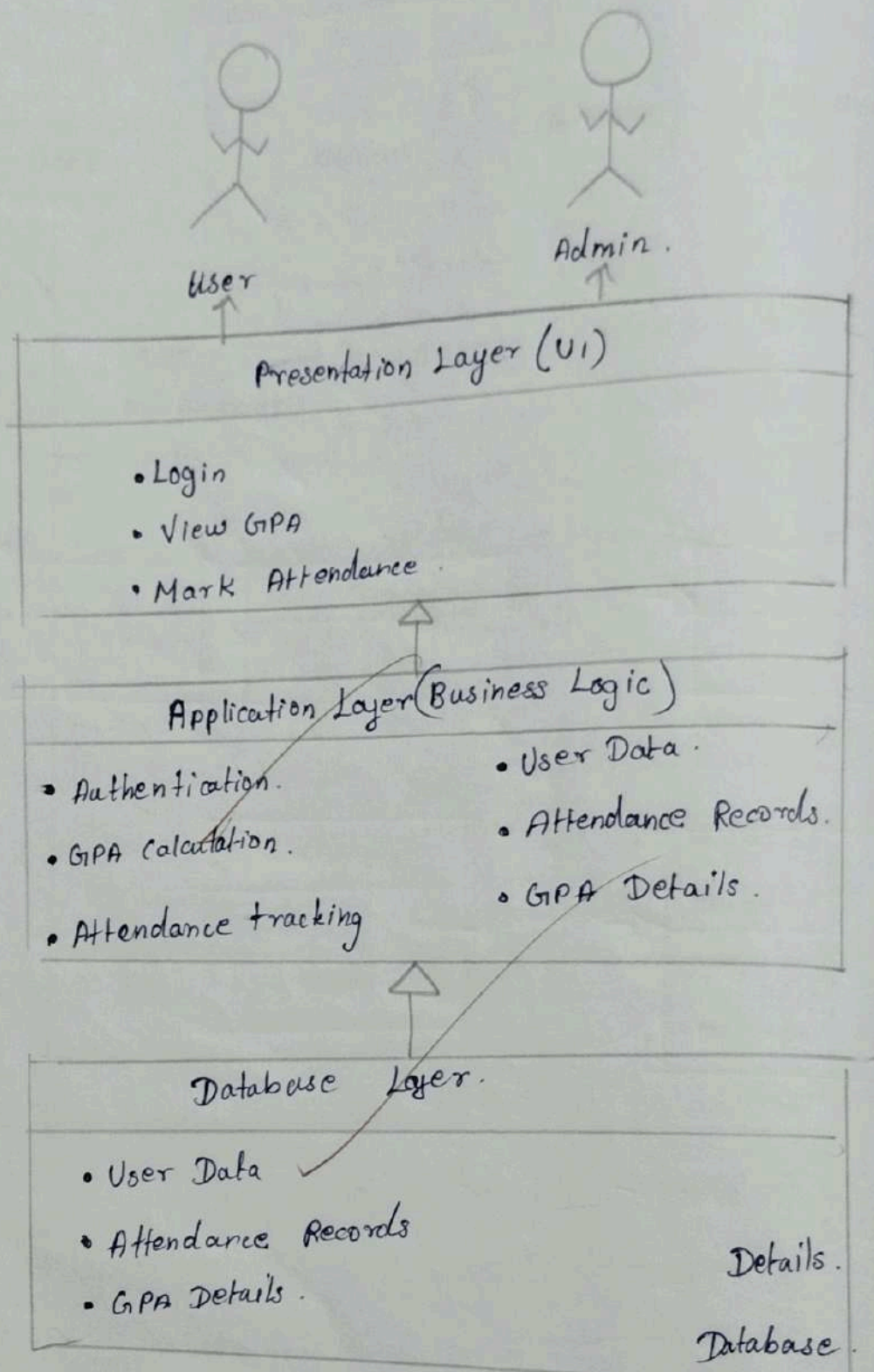
Sequence Diagram



RESULT:

The class and Sequence Diagrams were successfully designed, clearly representing both the structural and behavioral aspects of the system.

Architectural Diagram.



Ex. 4
7/4/26

DESIGNING ARCHITECTURAL AND ER DIAGRAMS

AIM :

To design Architectural and ER Diagrams for the given project to represent system structure and data organization.

OBSERVATION :-

Analyzed system requirements and identified components like UI, Application server, and Database.

Designed the Architectural Diagrams with layers: Presentation, Business Logic, and Data.

Identified entities such as User, Student, Faculty, Attendance, GPA, and Notification.

Created the ER Diagram showing entities, attributes, and relationships.

ARCHITECTURAL DIAGRAM DESCRIPTION :-

* Presentation Layer :

Handles user interactions like login, viewing GPA, and attendance.

* Business Logic Layer :

Processes operations such as authentication, GPA calculation, and attendance tracking.

* Data Layer :

Stores user data, attendance records, GPA details, and notifications.

ER DIAGRAM DESCRIPTION :-

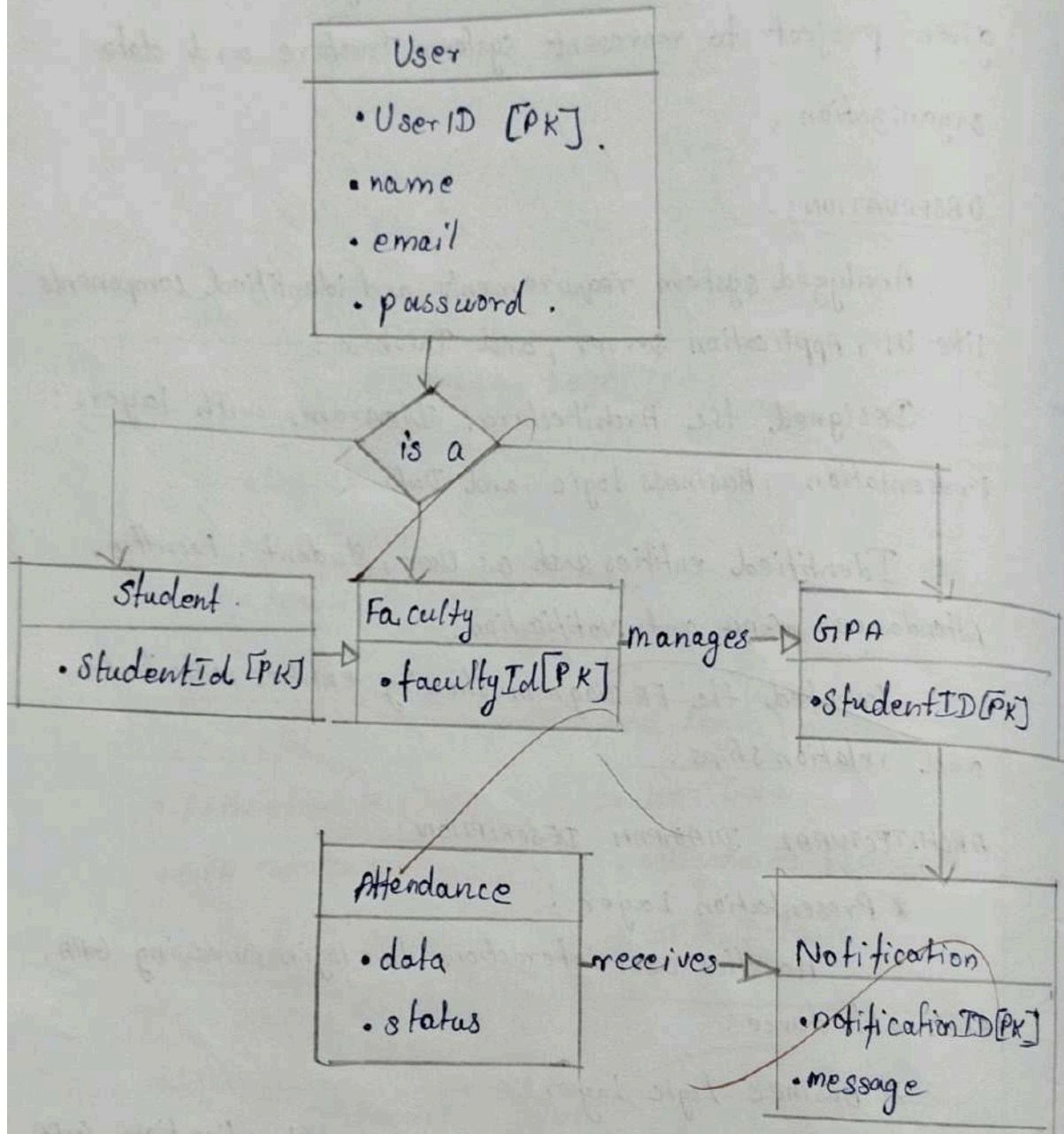
* Entities :

User (User ID, name, email, password)

Student (Student ID, GPA)

Faculty (Faculty ID)

ER Diagram



Attendance (attendance ID, date, status)

Notification (notification ID, message)

* Relationships:-

User can be Student or Faculty

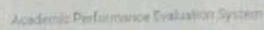
Student has GPA

Faculty manages Attendance

Student receives Notification

RESULT

The Architectural and ER Diagrams were successfully designed, clearly representing the system structure and relationships between data.



[Sign in to your account](#)

 Open Access

Don't have an account? [Sign up](#)

New documents are assigned the **Student** role by default.


AcadTrack
 Performance System

WELCOME

Dashboard

CPUA Evaluation
 Program Tracking
 Portfolio Tracking
 Analytics

Welcome back, Hannah

CS2022 Q147 - 8 Eych Computer Science

Overall GPA

3.54

 out of 4.00

Major GPA

3.58

 Corequisite

Total Credits

62

 20 completed

Latest GPA

3.45

 Semester A

Pending Work

Process Scheduling Simulation
Upcoming Semester - due 2024-01-19

Graph Algorithms Implementation
Due Semester - due 2024-01-09

Socket Programming Lab
Upcoming Semester - due 2024-09-10

Recent Alerts

Assignment Due Tomorrow
Process Scheduling Simulation - CS2022 Q147 due 2024-01-19

Low Performance Alert
Your overall is Computer Science is 3.45 below 3.50. Consider extra study.

Mid-Term Schedule Released
Upcoming Semester Mid-Term is published for class 1

Coming Up

Process Scheduling Simulation
Upcoming Semester - 2024-01-19

Graph Algorithms Implementation
Due Semester - 2024-01-09

SOC Quiz 2
Upcoming Semester - 2024-08-25

Ex. 8
16/4/26.

TESTING - TEST PLANS AND TEST CASES

AIM :-

To design Test Plans and Test Cases for the project to ensure proper system functionality and reliability.

OBSERVATION:

Analyzed the project modules such as Login system, Role Assignment, GPA Calculation, Attendance Tracking, and Notifications.

Prepared a Test Plan outlining testing objectives, scope, and strategies.

Identified different types of testing such as functional testing and validation testing.

Designed Test Cases for each module with inputs, expected outputs, and actual results.

Covered both valid and invalid scenarios to ensure system robustness.

Ensured all test cases are clear, complete, and easy to execute.

Test Plan Description:

- Objective : To verify that all modules work correctly.
- Scope : Login, GPA, Attendance, Notification modules.
- Testing Type : Functional Testing
- Approach : Execute test cases and compare expected vs actual results.

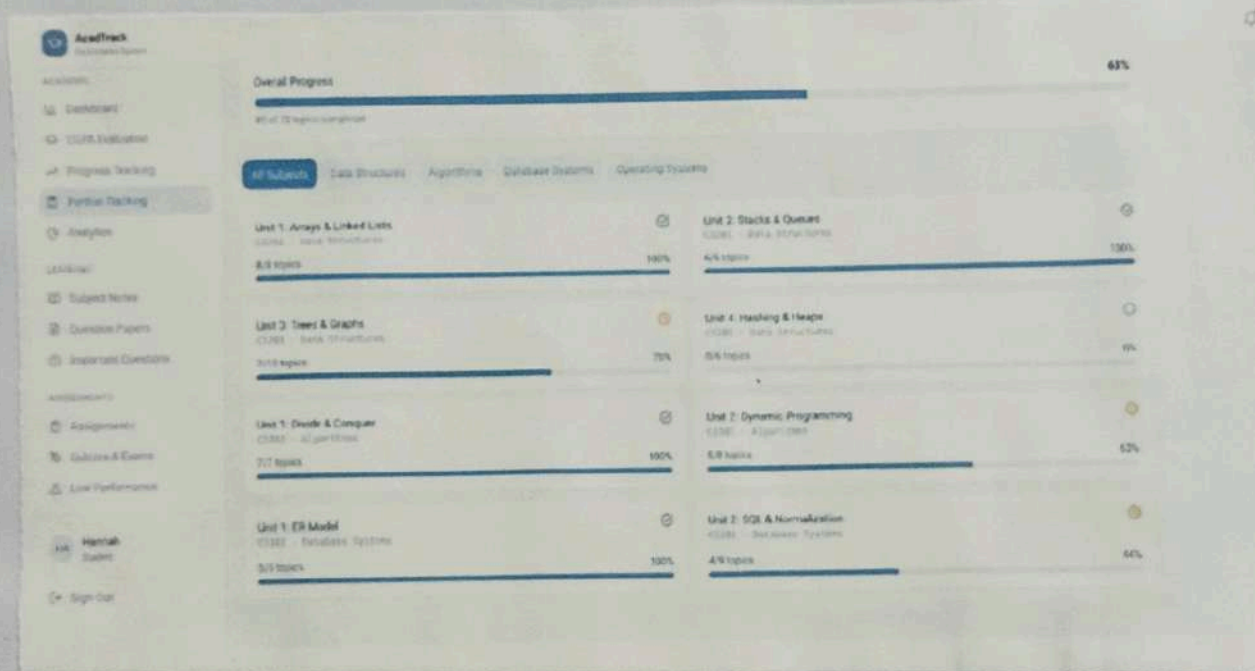
Test Case Description:

- Login Module :

Input : Valid / Invalid username & password

Expected Output : Login success or error message

- Role Assignment :



Input : Assign role to user

Expected Output : Role assigned correctly

- GPA Calculation :

Input : Student marks .

Expected Output : Correct GPA displayed

- Attendance Tracking :

Input : Mark attendance

Expected Output : Correct Attendance recorded successfully

- Notification System :

Input : Deadline or alert trigger

Expected Output : Notification sent to user.

RESULT :

The Test Plan and Test Cases were successfully designed, ensuring proper validation of all system modules and improving system reliability.

Academic Performance Evaluator

Application Insights

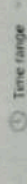
View in Logs



Stop



Refresh



Time range



Apps



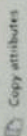
Copy link



Configure metrics



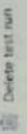
Download



Copy attributes



Share



Delete test run



Mark as baseline



Auto refresh On (10s)

Test run duration

3 hr

Time range

4/18/2025, 2:15:44 PM - 4/18/2025, 5:15:54 PM

Run type

Live Run

Engine instances

1

Debug mode

Disabled

Test result

Executing

Load test results

Engine health

Client-side metrics

Request: 4 selected

Regions: 0

Aggregation: P90

Error type: 5 selected

Time range: 4/18/2025, 2:15:44 PM - 4/18/2025, 5:15:54 PM

Group by: %

Virtual Users (Max)

80

60

40

20

0

Max

48

Request

Time

8000

6000

4000

2000

0

Requests (per second) (successful responses)

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

Response time (Avg)

Max

48

1 /sec

Errors (total)

1

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

Exp 9
16/4/26

LOAD TESTING AND PIPELINES

AIM:

To Perform Load Testing and implement Pipelines for continuous integration and deployment.

OBSERVATION:

Analyzed System performance requirements and identified Key modules for load testing such as login, GPA, and Attendance.

Simulated multiple user requests to test system behaviour under heavy load.

Monitored response time, throughput, and system stability.

Identified performance bottlenecks and ensured system reliability.

Setup pipelines for automating build, test, and deployment processes.

Configured steps like code integration, testing, and deployment in the pipeline.

Ensured smooth and continuous delivery of the application.

LOAD TESTING DESCRIPTION:

Tested system with multiple users simultaneously.

Checked response time and performance

Ensured system handles high traffic without failure.

PIPELINE DESCRIPTION:-

Automated build and testing process.

Continuous integration of code changes.

Continuous Deployment of application.

Academic Performance Evaluator

Application insights

View in logs ☐ Stop ☐ Refresh ☐ Time range: 1 hr

Test run duration: 3 hr

Time range: 4/18/2025, 2:15:44 PM - 4/18/2025, 5:15:14 PM

Load test results Engine health

Client side metrics

Requests: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

Search resources, services, and docs (Ctrl+K)

Capitol

View in logs ☐ Stop ☐ Refresh ☐ Time range: 1 hr

Test run duration: 3 hr

Time range: 4/18/2025, 2:15:44 PM - 4/18/2025, 5:15:14 PM

Load test results Engine health

Client side metrics

Requests: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

Auto refresh On (10s)

Test result

Engine health

Client side metrics

Requests: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

App: 4 selected

RESULT :

Load Testing was successfully performed,
and pipelines were implemented to ensure
system performance and continuous delivery.

github.com/Hannah-241001069/Academic-Performance-Evaluator

Hannah-241001069 / Academic-Performance-Evaluator

Type to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Academic-Performance-Evaluator

Public

main 1 branch 0 tags

Go to file

Add file Code

Monica

4:23PM 34 Commits

Hannah-241001069	Update README.md	4:23PM	34 Commits
241001069	diagram	Add files via upload	3 weeks ago
Diagram 241001069		Add files via upload	3 weeks ago
Load Testing		Add files via upload	last week
Pipelines		Add files via upload	17 minutes ago
Poker Estimation		Add files via upload	3 weeks ago
Progress Report		Add files via upload	last week
Sprints		Add files via upload	3 weeks ago
Testing Test Plans and Test Cases		Add files via upload	last week
Azure account/pt		Add files via upload	2 months ago
project	Create requirements.txt		51 minutes ago
static	Add files via upload		2 days ago
templates	Add files via upload		2 days ago
backlog pic.png	Rename Screenshot (227) png to Backlog pic.png		1 month ago
README.md	Update README.md		new
azure-pipelines.yml	Update azure-pipelines.yml for Azure Pipelines		28 minutes ago

README

Repo Summary

Supports the most advanced mode to help you quickly understand the contents of the repo

Summarize this repo

About

Academic-Performance-Evaluator is a powerful tool to analyze and manage academic progress. It evaluates performance metrics, provides insights into grade distributions, and supports academic goal setting based on historical data. It offers clear visualization of student academic standing.

academic evaluator grades analysis
education-data data-analysis
azure-dwops machine-learning
academic-app python

Readme
Activity
5 stars
1 watching
0 forks

Handwritten notes on lined paper, mostly illegible due to blurriness. Some visible text includes:

- ... of the ...
- ... of the ...
- ... of the ...
- ... of the ...

CONVENTIONSAIM:

To organize the project using proper structure and naming conventions in GitHub.

OBSERVATION:

Create a repository in GitHub for the project

Organized files into folders such as src, docs, and tests.

Maintained proper naming conventions for files, variables, and functions.

Used meaningful and consistent names for better readability.

Added README file to describe the project details.

Managed version control using commits and updates.

Ensured clean and well-structured project organization.

PROJECT STRUCTURE DESCRIPTION:

src : Source Code files

docs : Documentation files

tests : Test cases and testing files.

NAMING CONVENTIONS:

Used meaningful and descriptive names

Followed consistent format (camel Case / Pascal Case)

Avoided unnecessary abbreviations.

RESULT :

The project was successfully organized in GitHub with proper structure and naming conventions, improving readability and maintainability.